

TECHCATALYST DE 2026

Reliable Delta Pipelines and Lakeflow Jobs

Use history, safe reruns, layer contracts, validation, and repair to make a pipeline operable.

Week 5
Day 3

Reliable tables become an operable pipeline.

Scheduling raises the standard of correctness

A notebook can work once

A human can run cells out of order, repair state, edit a value, or overlook a warning.

The output may still look correct.



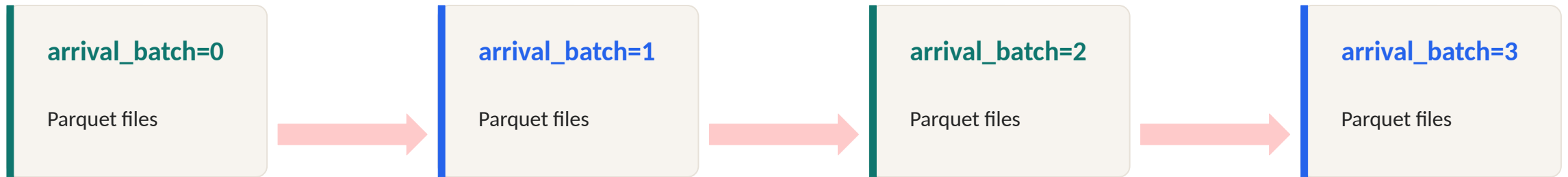
A pipeline must explain itself

Inputs and outputs are explicit. Reruns are safe. Contracts fail loudly. Every task has history and ownership.

Operational quality is visible in the run graph, table history, and validation evidence.

Volumes keep path-based data inside Unity Catalog

/Volumes/workspace/your_schema/landing/taxi_batches



One folder read

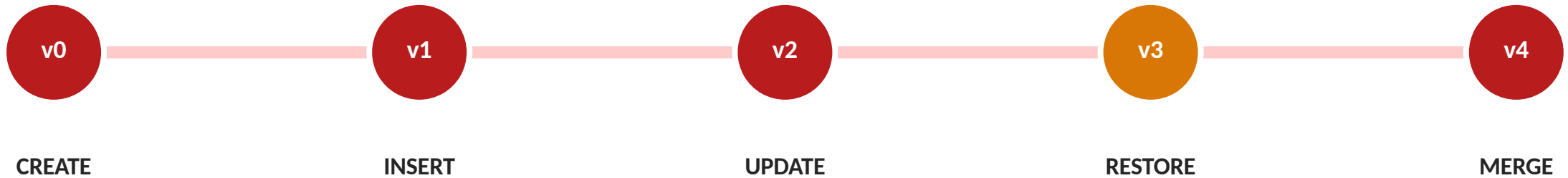
Spark recovers partition columns and _metadata preserves source lineage.

01

The Delta log records every table state

History makes debugging, audit, time travel, and restore possible.

Each successful write commits a new snapshot



DESCRIBE HISTORY shows the operation, timestamp, version, and useful operation metrics.

Time travel reads the past. Restore changes the present.

VERSION AS OF

Question

What did the table look like then?

Effect

Reads a prior snapshot. Current state stays unchanged.

Use

Audit, debug, compare, reproduce

RESTORE TABLE

Question

Should that prior data become current again?

Effect

Creates a new current version. History remains.

Use

Operational recovery after evidence review

Inspect history first. Hardcoded version numbers break after reruns.

Idempotency means the same batch converges on the same state

Unsafe append

Batch A arrives

Run 1: rows added

Run 2: rows added again

Same input, different target state

Conditional MERGE

Match on a stable key

Update only changed values

Insert only missing entities

Same input, same target state

Proof

Capture sorted state

Run the same MERGE again

Capture state again

Assert equality

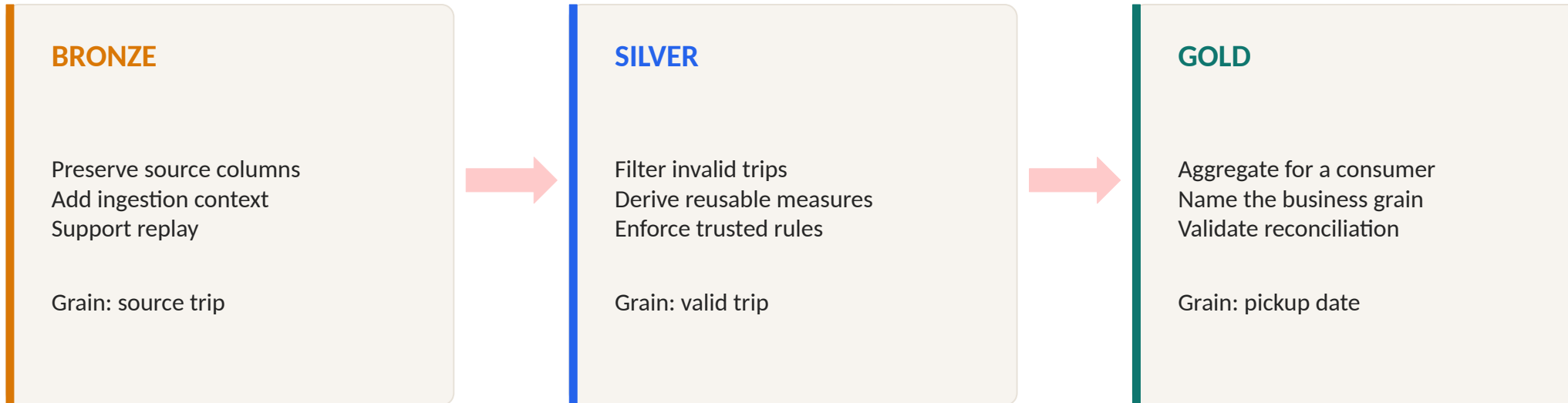
A rerun that succeeds but duplicates data is not idempotent.

02

Layer contracts make the pipeline explainable

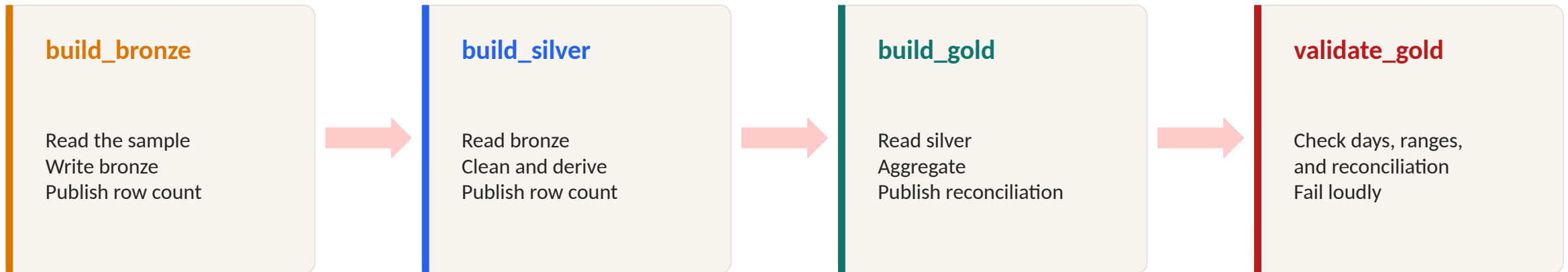
Bronze preserves, silver enforces, and gold serves one declared grain.

Each layer earns its place through a contract



SUM(gold.total_trips) = COUNT(silver trips)

The task graph makes order and failure boundaries visible



Serverless task state is isolated. Unity Catalog tables carry the durable data.

Free Edition limit: design compact jobs and avoid unnecessary concurrent tasks.

Use the right channel for each kind of information

%run

Shared setup code

Scope
One notebook context

Example
Validated catalog and schema
names

Job parameter

Configuration

Scope
Provided at run time

Example
target_schema or batch_id

Table

Durable dataset

Scope
Across tasks and runs

Example
silver_taxi

Task value

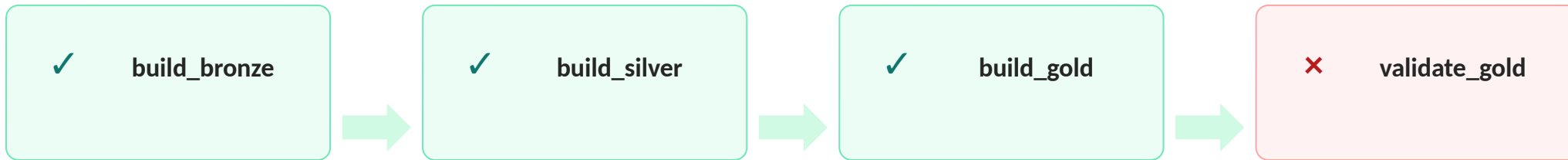
Small run metadata

Scope
This job run

Example
silver_rows

Do not serialize a DataFrame into a task value. Do not create a table for one counter.

Repair the failed boundary, not every successful task



Evidence

The build outputs reconciled. The failure says `minimum_days` is 9999. The parameter changed, not the data or code.

Targeted response

Correct the parameter. Repair the failed task and downstream work. Preserve the successful build evidence.

Wanderbricks turns the patterns into a data product

Discover

Inspect bookings, payments, properties, and users. Declare keys, grains, statuses, and risks.

Build and operate

Create bronze copies, one booking-grain silver fact, two gold outputs, and a validated Lakeflow Job.

Defend

Show the architecture, a green run, a repaired failure, two SQL queries, and one dashboard insight.

Design first. Build second. Validate always. Defend the trade-offs.